AI with Rav

Learn AI the way you actually think.

DAY 4 of 30

PYTHON

Numbers & Math — the calculator built in

WHAT YOU'LL LEARN TODAY

- The everyday operators
- The special three (// % **)
- int vs float
- Order of operations
- The += shortcut

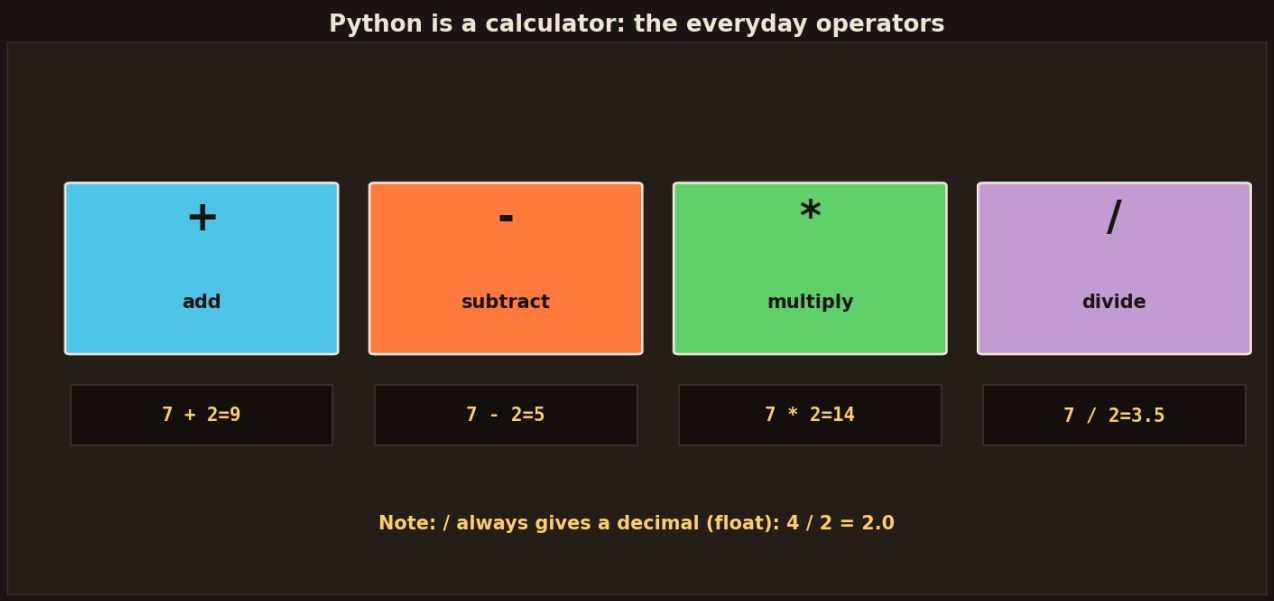
Numbers & Math — the calculator built in

> In One Line: Python is a powerful calculator built right in. You can add, subtract, multiply, divide — plus a few special operators for "whole-number division", "remainder", and "power". Numbers are the heart of all AI, so this is core.

Start here: Python does your maths

AI is built on numbers — every model is really just a giant pile of arithmetic. So Python makes maths effortless. You can type sums straight into it and it just calculates, no special setup. Today we cover the operators you'll use every day, plus three special ones that quietly solve real problems.

The everyday operators

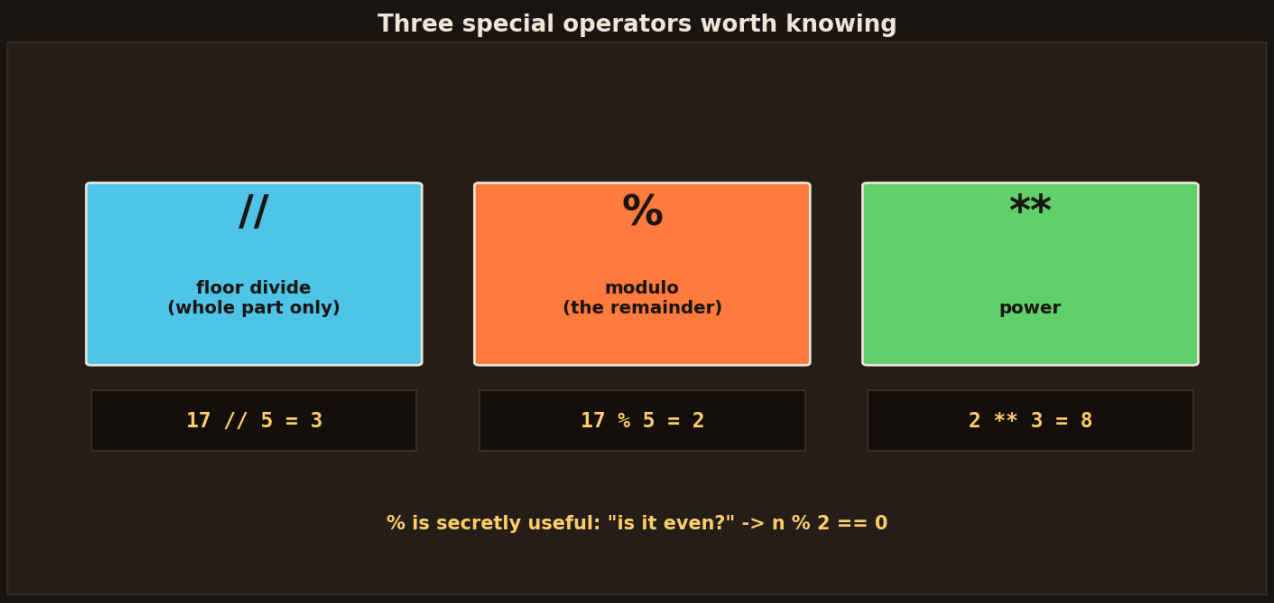


The four everyday math operators: + (add), - (subtract), * (multiply), / (divide). Example: 7 + 2 = 9, 7 * 2 = 14, 7 / 2 = 3.5. Note that / always gives a decimal.

```
print(7 + 2)     # -> 9    add
print(7 - 2)     # -> 5    subtract
print(7 * 2)     # -> 14   multiply  (star, not x)
print(7 / 2)     # -> 3.5  divide
```

> Two things to notice: multiply is `*` (a star, never `x`), and **divide `/` always gives a decimal** — even `4 / 2` gives `2.0`, not `2`. That surprises beginners, but it's consistent: division might not come out even, so Python always returns a decimal (float) to be safe.

The special three: // % **



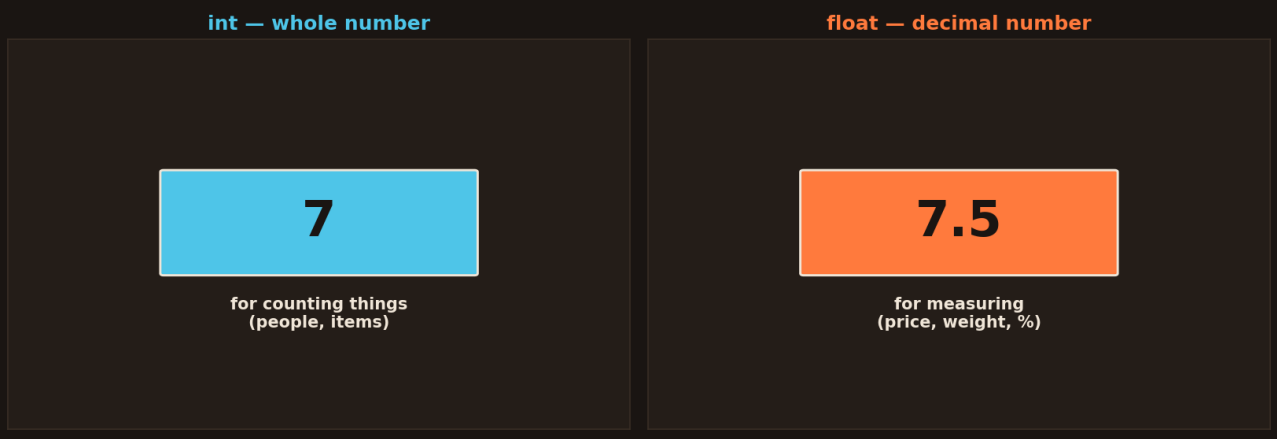
Three special operators: // floor divide (17 // 5 = 3, the whole part only), % modulo (17 % 5 = 2, the remainder), and ** power (2 ** 3 = 8). The % operator is secretly useful for checking if a number is even.

These three look odd but are genuinely useful:

- `//` **floor divide** → the whole-number part only: `17 // 5` = `3` (drops the leftover).
- `%` **modulo** → the **remainder** after dividing: `17 % 5` = `2`.
- `` power → **raise to a power: `2** 3`** = `8` (2×2×2).

> `%` (modulo) looks useless but is secretly one of the most-used operators. The classic trick: **"is this number even?"** → `n % 2 == 0`. If the remainder when dividing by 2 is 0, it's even. It's used everywhere — "every 5th item", "which day of the week", wrapping around a clock. Keep it in your back pocket.

int vs float — whole vs decimal



int holds a whole number like 7 (for counting things — people, items). float holds a decimal like 7.5 (for measuring — price, weight, percentages).

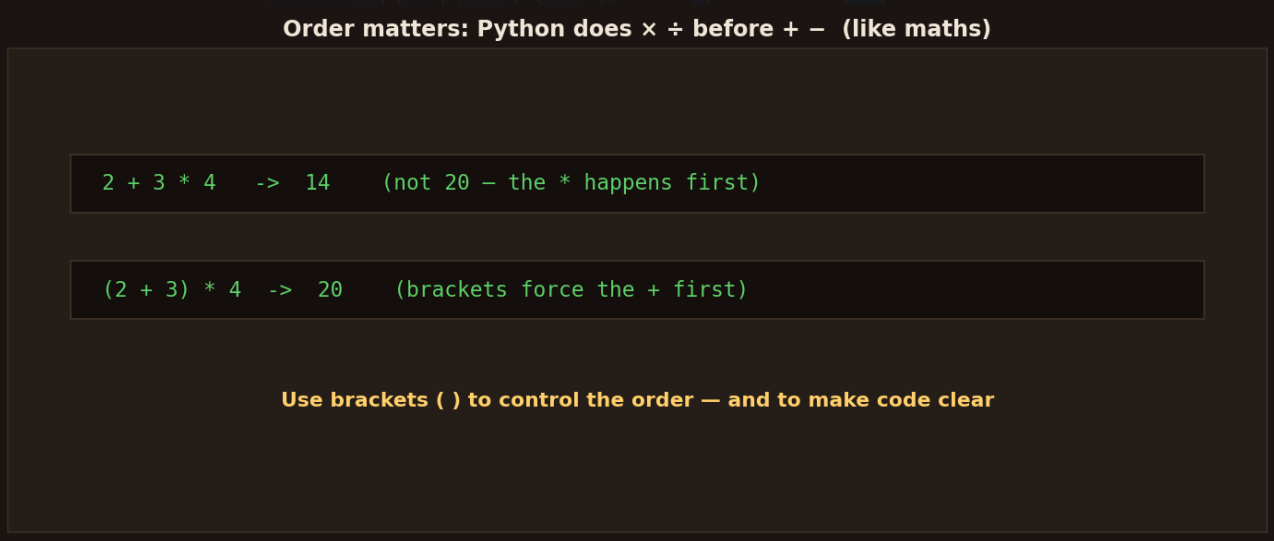
Remember the two number types from Day 2:

- **int** → whole numbers (`7`), for **counting** — people, items, clicks.
- **float** → decimal numbers (`7.5`), for **measuring** — price, weight, percentages.

```
count = 7          # int
price = 7.5        # float
print(count + price)    # -> 14.5   (mixing them gives a float)
```

> You can freely mix them in maths — `7 + 7.5` works and gives `14.5`. Notice the answer becomes a float: whenever a decimal is involved, the result is a decimal. Division `/` also always makes a float. This "safety" behaviour means you rarely lose the fractional part by accident.

Order of operations



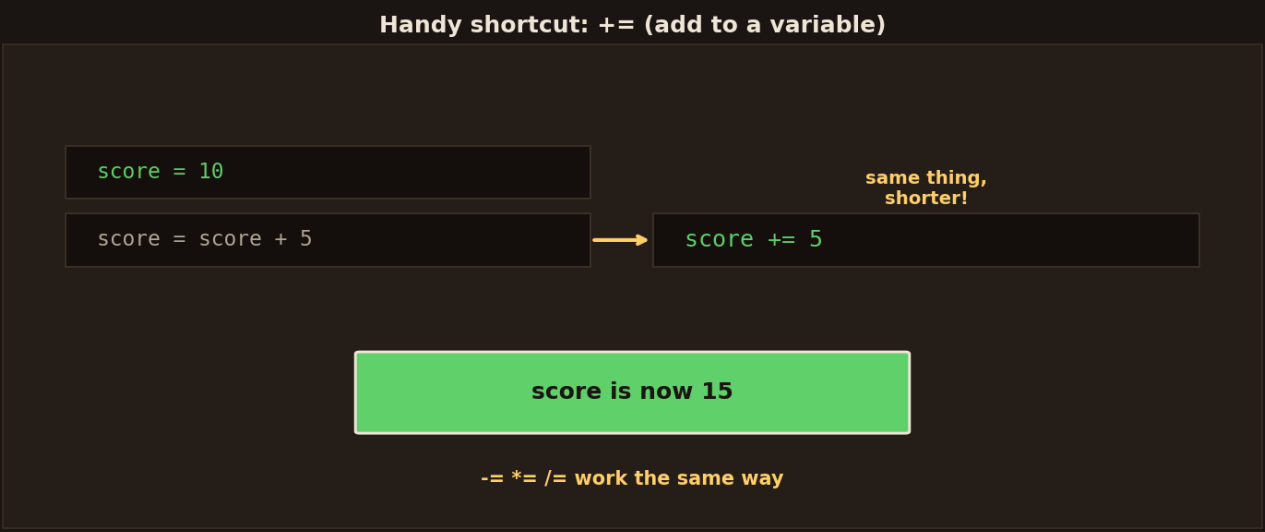
Order matters: 2 + 3 * 4 gives 14 (the * happens first, not 20). But (2 + 3) * 4 gives 20 — brackets force the + to happen first.

Python follows the same maths rules you learned in school — multiply and divide happen **before** add and subtract:

```
print(2 + 3 * 4)      # -> 14   (3*4 first, then +2)
print((2 + 3) * 4)    # -> 20   (brackets force 2+3 first)
```


When in doubt, **use brackets** `( )` to force the order you want — and to make your intention obvious to anyone reading. Even when brackets aren't strictly needed, they make code clearer. Clarity beats cleverness every time.

The += shortcut



A handy shortcut: instead of score = score + 5, write score += 5 — same thing, shorter. If score was 10, it's now 15. The same works for -=, *=, /=.

You'll often want to *update* a variable using its own value — like adding to a running total:

```
score = 10
score = score + 5     # the long way
score += 5            # the shortcut — same thing!
print(score)          # -> 20
```

`score += 5` means "add 5 to whatever score already is." It's just a shorter way to write `score = score + 5`. The same shortcut works for the others: `-=`, `*=`, `/=`. You'll use `+= 1` constantly to count things — it's the bread and butter of loops (coming soon).

Recap — the 20-second version

- Everyday operators: `+ - * /` — and `/` always gives a **decimal** (float).
- Special three: `//` (whole-part divide), `%` (**remainder**), `**` (power).
- `%` is secretly gold: `n % 2 == 0` checks if a number is **even.**
- Maths order applies (× ÷ before + –); use **brackets** to control it.
- `score += 5` is the short way to write `score = score + 5`.

Next up — Video 5: Input, Output & Comments. Let's make our programs interactive — asking the user a question and using their answer — and learn to write clean, commented code that your future self will understand. See you tomorrow.